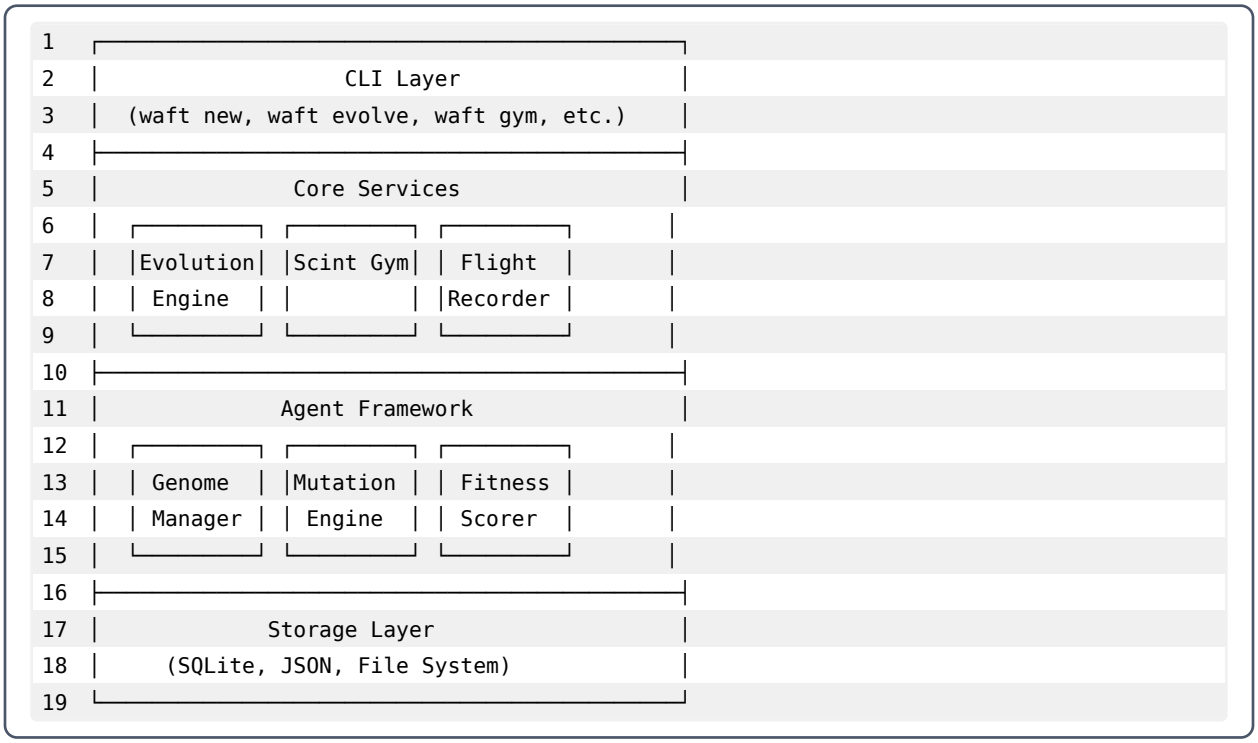
WAFT ARCHITECTURE

Technical Overview for Developers

System Overview

WAFT is built on a modular architecture designed for extensibility and scientific rigor.



Core Components

Evolution Engine

The heart of WAFT. Manages populations, runs selection, and coordinates breeding.

```python
class EvolutionEngine:
    def __init__(self, config: EvolutionConfig):
        self.population: list[Agent] = []
        self.generation: int = 0
        self.flight_recorder = FlightRecorder()

    def run_generation(self) -> GenerationResult:
        # 1. Evaluate all agents
        scores = self.evaluate_population()
```

```
10
11          # 2. Select survivors
12          survivors = self.select(scores)
13
14          # 3. Breed next generation
15          offspring = self.breed(survivors)
16
17          # 4. Apply mutations
18          mutated = self.mutate(offspring)
19
20          # 5. Record and return
21          self.flight_recorder.log_generation(...)
22          return GenerationResult(...)
```

Scint Gym

The fitness evaluation environment.

```python
class ScintGym:
    def __init__(self):
        self.scenarios: list[Scenario] = []
        self.detector = RegexScintDetector()

    def evaluate(self, agent: Agent) -> FitnessScore:
        results = []
        for scenario in self.scenarios:
            # Run agent against scenario
            response = agent.process(scenario.prompt)

            # Detect any Scints
            scints = self.detector.detect(response)

            # Score the result
            score = self.score(scenario, response, scints)
            results.append(score)

        return FitnessScore.aggregate(results)
```

Flight Recorder

Complete telemetry for all evolutionary events.

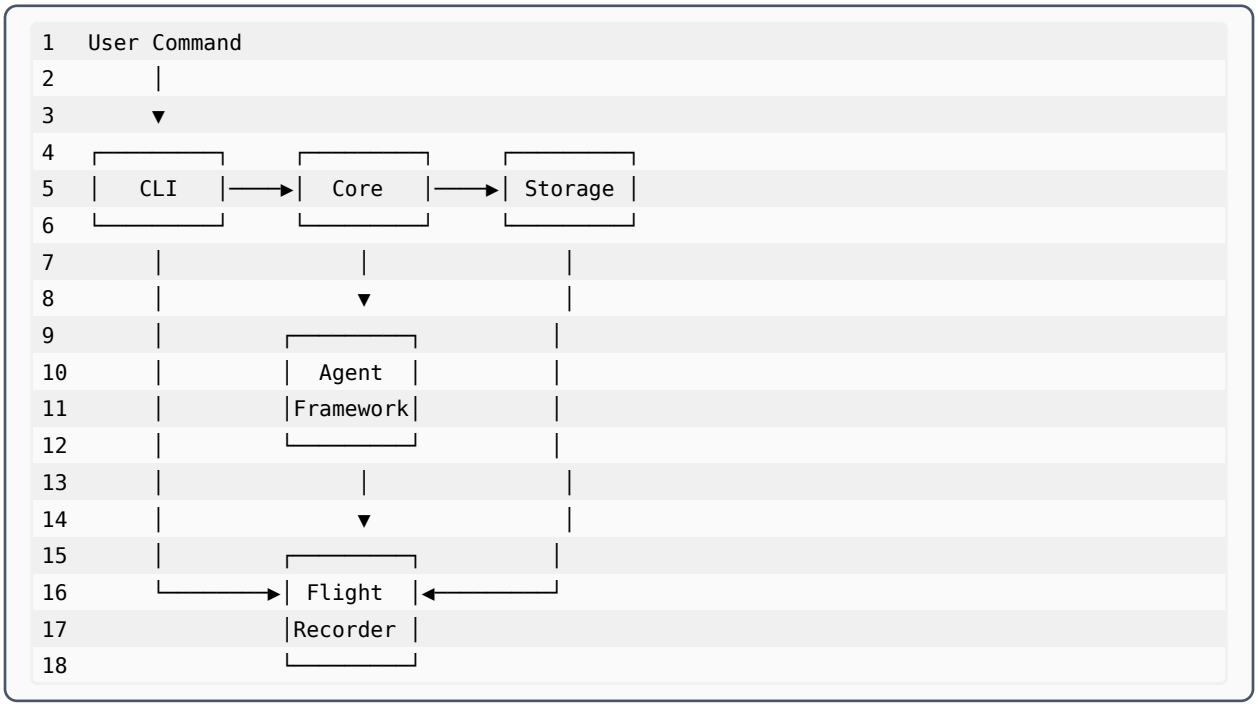
```python
class FlightRecorder:
    def __init__(self, db_path: Path):
        self.db = sqlite3.connect(db_path)

    def log_event(self, event: Event) -> str:
        event_id = generate_event_id()
        self.db.execute("""
            INSERT INTO events
            (id, timestamp, type, genome_id, metadata)
            VALUES (?, ?, ?, ?, ?)
        """, (event_id, event.timestamp, ...))
        return event_id

    def query(self, **filters) -> list[Event]:
        # Flexible querying by type, time, genome, etc.
        ...
```

Directory Structure

```
waft/
├── src/waft/
│   ├── cli/           # Command-line interface
```

```
│   ├── core/          # Core business logic
│   │   ├── evolution/ # Evolution engine
│   │   ├── gym/       # Scint Gym
│   │   └── agents/    # Agent framework
│   ├── storage/       # Persistence layer
│   └── utils/         # Shared utilities
├── tests/             # Test suite
├── docs/              # Documentation
└── examples/          # Example usage
```

Data Flow



Configuration

waft.toml

```toml
[project]
name = "my_evolution_lab"
version = "0.1.0"

[evolution]
population_size = 20
mutation_rate = 0.1
selection_pressure = 0.5

[gym]
timeout_seconds = 300
parallel_workers = 4
```

```
scenarios = ["syntax", "logic", "safety", "hallucination"]

[storage]
database = "waft.db"
checkpoints_dir = "checkpoints/"
```

Extension Points

Custom Agents

```python
from waft import Agent

class MyAgent(Agent):
    def process(self, input: str) -> str:
        # Your implementation
        return result
```

Custom Mutations

```python
from waft.mutations import Mutator

class MyMutator(Mutator):
    def mutate(self, genome: Genome) -> Genome:
        # Your mutation logic
        return mutated_genome
```

Custom Fitness Functions

```python
from waft.gym import FitnessFunction

class MyFitness(FitnessFunction):
    def score(self, agent: Agent, scenario: Scenario) -> float:
        # Your scoring logic
        return score
```